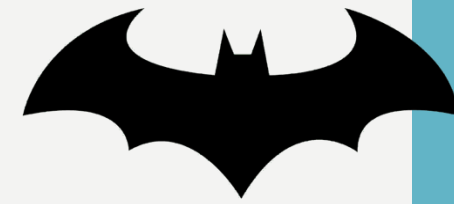




TP2 – LE SYSTÈME DE BADGES DE LA BATCAVE

PROTOCOLES D'AUTHENTIFICATION

CONTEXTE



Le système *Basic Auth* mis en place lors du TPI fonctionne, mais Bruce Wayne a rapidement identifié ses limites :

1. Les identifiants circulent à chaque clic sur le réseau ;
2. Alfred est incapable de révoquer l'accès d'un appareil à distance sans changer globalement le mot de passe de l'utilisateur.

Alfred veut moderniser l'infrastructure !

👉 Désormais, le justicier se connecte, le serveur valide, lui attribue un **badge éphémère (un Session Id via un cookie)** et mémorise ce badge.

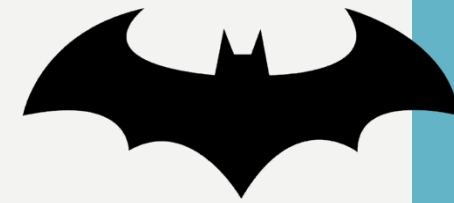


INSTRUCTIONS

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

ÉTAPE 1 : REFACTO



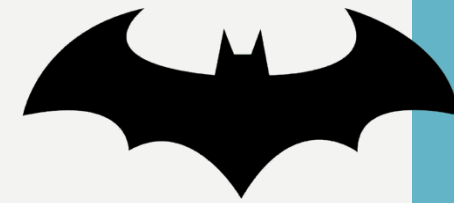
Avant d'ajouter les sessions, nous devons casser le fichier unique `server.js` du TPI qui va devenir illisible à force d'y ajouter du code !

Pour ce faire, vous devez isoler les responsabilités selon l'arborescence suivante :

```
arborescence

/batcave-security
├── /config          # Initialisation de la base SQLite (ex db.js)
├── /middlewares     # Les vigiles de sécurité (ex: checkAuth, checkRole)
├── /routes          # Les fichiers de routes par entité (auth.js, batcomputer.js)
├── /views           # Les fichiers HTML de vos interfaces (hors du dossier public !)
├── /public          # Uniquement les fichiers CSS et JS accessibles à tous
├── .env             # Vos secrets d'infrastructure (À NE JAMAIS PUSH !)
├── .gitignore       # Protection contre les fuites
├── server.js        # Point d'entrée léger (uniquement les imports et app.listen)
└── package.json
```

ÉTAPE 1 : REFACTO



1. Le **point d'entrée** (`server.js`) ne doit plus contenir aucune route. Son rôle est uniquement d'importer Express, le routeur et de lancer le serveur.
2. Dans `routes/auth.js`, créez votre **routeur Express** :

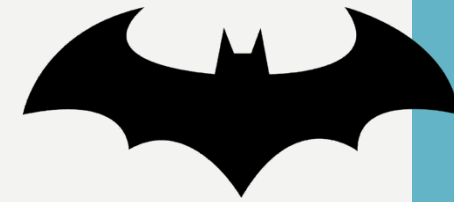
```
routes/auth.js

const express = require('express');
const router = express.Router();
// Vos routes de connexion ici...
module.exports = router;
```

[Copier le code](#)

Dans `server.js`, vous l'importerez et l'utiliserez => `app.use('/auth', authRouter);`.

ÉTAPE 2 : SÉCURISATION DE L'INFRASTRUCTURE



Installez le module `dotenv` pour externalisez la clé de signature de vos cookies ainsi que le port d'écoute du serveur dans le fichier `.env` (à créer à la racine de votre projet).

Stockez-y votre port (`PORT=3000`) et une clé aléatoire pour signer vos sessions (`SESSION_SECRET=votre_cle_super_secrete`)

Installez le middleware `express-session`. Configurez-le dans le fichier `server.js` en y injectant votre **variable d'environnement** `process.env.SESSION_SECRET`.

Le cookie généré par Express doit obligatoirement être configuré avec les **options (flags) de sécurité** maximales et renommé pour masquer la technologie utilisée :

- Nom du cookie : `bat_identity`
- `httpOnly: true` pour interdire l'accès au cookie via les scripts JS du client ;
- `sameSite: 'strict'` pour que le cookie ne soit pas envoyé si la requête provient d'un site tiers => **Anti-CSRF** ;
- `maxAge: 1800000` pour déconnecter automatiquement l'utilisateur après 30 minutes.

ÉTAPE 3 : LE PARCOURS UTILISATEUR ET LA LOGIQUE DE SESSION

Vous devez maintenant implémenter les **4 routes** clés du cycle de vie de l'authentification :

1. Le formulaire de connexion (GET /auth/login)

👉 Cette route doit renvoyer un document HTML propre contenant un formulaire avec deux champs : un **e-mail** (ou un nom d'utilisateur) et un **mot de passe**.

💡 N'oubliez pas d'ajouter le middleware `app.use(express.urlencoded({ extended: true }))` à votre serveur pour pouvoir lire les données issues de votre formulaire HTML.

2. Le traitement de la connexion utilisateur (POST /auth/login)

1. **Vérification** : Le serveur récupère les données du formulaire, interroge la table `users` et compare le mot de passe avec `bcrypt.compare()` ;
2. **Fixation de session** : Si le mot de passe est correct, vous devez détruire l'ancienne session éphémère et en régénérer une nouvelle pour éviter qu'un pirate n'ait fixé l'ID à l'avance.

ÉTAPE 3 : ALGORITHME À SUIVRE

```
routes/auth.js

req.session.regenerate((err) => {
  if (err) next(err);
  // 1. Stocker l'email ou le username dans la nouvelle session :
  req.session.user = { username: ... }
  // 2. Sauvegarder explicitement : req.session.save()
  // 3. Rediriger l'utilisateur vers le tableau de bord (/bat-computer)
});
```

ÉTAPE 3 : LE PARCOURS UTILISATEUR ET LA LOGIQUE DE SESSION

3. La route protégée (GET /bat-computer)

- Créez un middleware `isAuthenticated()` dans le fichier `middlewares/authCheck.js` chargé de vérifier si `req.session.user` existe. Si non, il renvoie un code `401` et redirige sur la page de connexion ;
- Appliquez ce middleware à la route protégée `/bat-computer`. Cette route doit renvoyer un document HTML personnalisé affichant un message de bienvenue incluant **l'email ou le nom d'utilisateur** de l'agent stocké dans sa session.

4. La déconnexion propre (GET /auth/logout)

- En plus de détruire la session, la route doit également effacer le cookie du navigateur ;
- Redirigez ensuite l'agent vers la page de connexion.

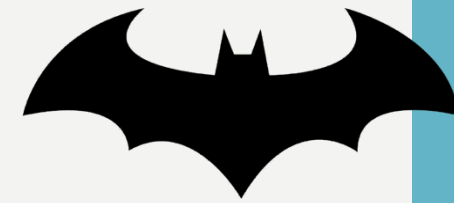


POUR ALLER PLUS LOIN

CHRIS CHEVALIER

CHEVALIER@CHRIS-FREELANCE.COM

ÉTAPE BONUS 1



Seulement pour les étudiants ayant terminé les phases précédentes !

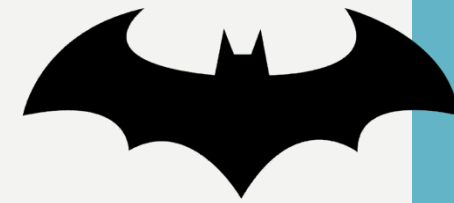
Par défaut, `express-session` stocke les sessions en RAM.

👉 Si le serveur de la Batcave redémarre ou crash, Bruce Wayne est déconnecté !

Pour palier ce problème, configurez un **Session Store** persistant pour que les sessions d'`express-session` soient automatiquement écrites dans une table dédiée de votre base de données SQLite.

Prouvez lors de votre code que si vous coupez le serveur (Ctrl+C) et que vous le relancez, votre session utilisateur est toujours active sur le navigateur.

ÉTAPE BONUS 2



Seulement pour les étudiants ayant terminé les phases précédentes !

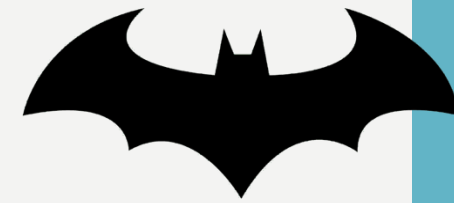
Un pirate parvient à voler le cookie `bat_identity` de Bruce Wayne (ex: via une clé USB physique). S'il le colle dans son propre navigateur, il peut pénétrer dans le système du Bat-ordinateur !

Lors du login, enregistrez dans `req.session` l'adresse IP de l'utilisateur ET son agent utilisateur (l'empreinte de son navigateur via `req.headers['user-agent']`).

Créez un **middleware de sécurité** qui, à chaque requête, compare l'adresse IP et l'agent utilisateur actuels avec ceux stockés dans la session.

👉 Si les empreintes ne correspondent pas (changement suspect d'appareil ou d'adresse IP en cours de session), le serveur doit déclencher une **alerte**, détruire immédiatement la session et bloquer l'accès.

ÉTAPE BONUS 3



Seulement pour les étudiants ayant terminé les phases précédentes !

Alfred exige une **traçabilité totale** sur les accès aux infrastructures sensibles.

Pour ce faire, créez une table SQLite `connexions_audit`.

👉 Chaque événement de connexion réussie, déconnexion volontaire ou tentative de fraude (empreinte suspecte du bonus 2) doit y être consigné !

💡 Données à stocker : `username`, `action` (`LOGIN`, `LOGOUT` ou `FRAUD`), `ip_address`, `user_agent` et `timestamp`.

👉 Une page d'administration réservée aux administrateurs doit permettre de consulter l'**historique de ces logs** sous forme de tableau propre.



DES QUESTIONS ?

CHEVALIER@CHRIS-FREELANCE.COM